



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊

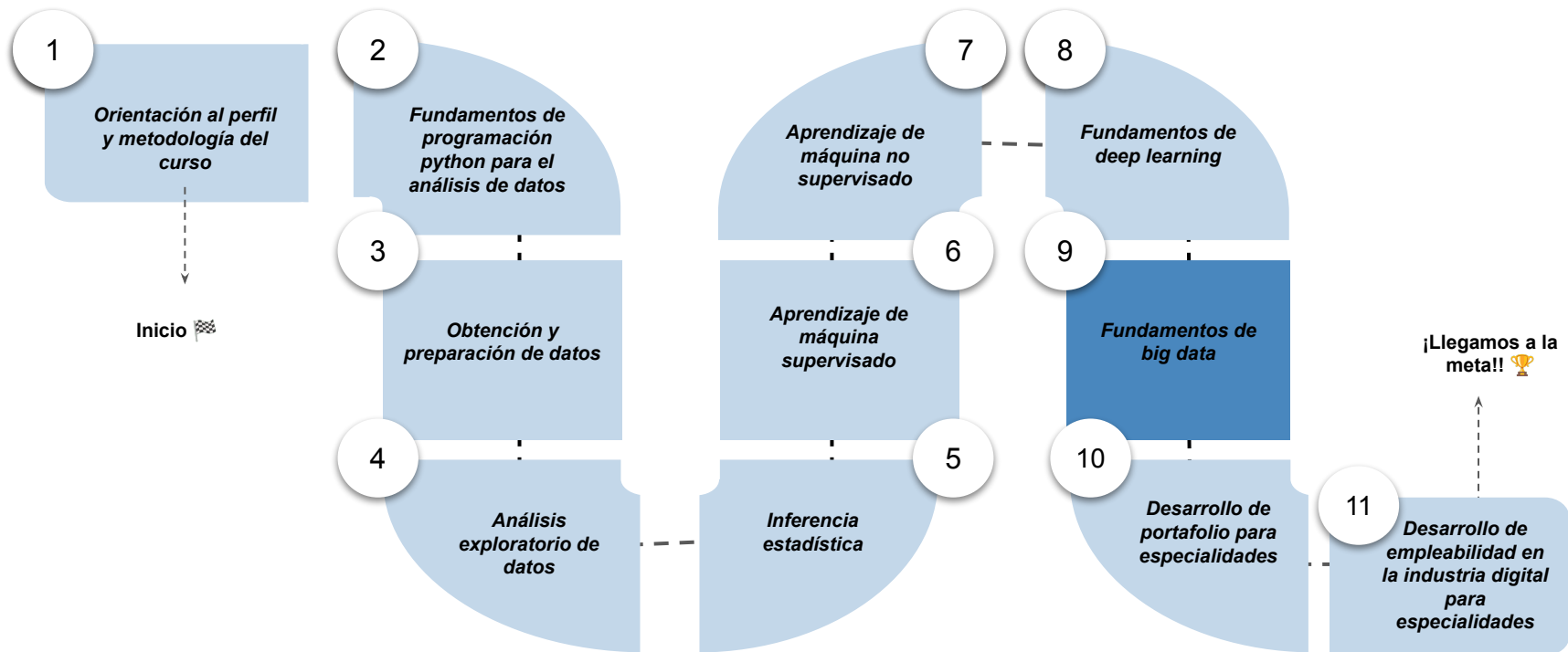


› Introducción a machine learning escalable

Aprendizaje Esperado 5: Elaborar un modelo de machine learning (aprendizaje de máquina) utilizando mllib de apache spark para resolver un problema de grandes volúmenes de datos.

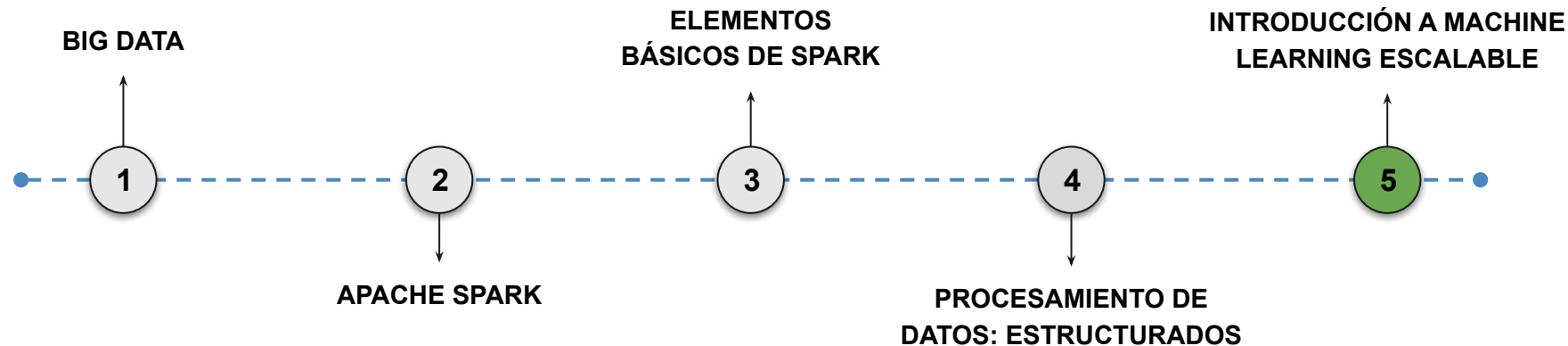
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Fundamentos de MLlib y su ecosistema en Spark

Introduciremos MLlib, la biblioteca de Machine Learning de Apache Spark, explorando sus características, estructuras de datos y algoritmos, con foco en cómo aprovecharla para grandes volúmenes de datos.

Conceptos y características de MLlib

¿Qué es MLlib y por qué es escalable?

Integración con el ecosistema Spark.

Estructuras de datos y herramientas en MLlib

DataFrames, vectores y matrices.

Pipelines de Machine Learning.

Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Implementación de
algoritmos supervisados y
no supervisados con MLlib

Implementaremos modelos de Machine Learning en Spark MLlib, aplicando algoritmos supervisados y no supervisados, desde la preparación de datos hasta la evaluación.

Implementación de
algoritmos supervisados

Flujo de trabajo: preparación, definición, entrenamiento, predicción, evaluación.

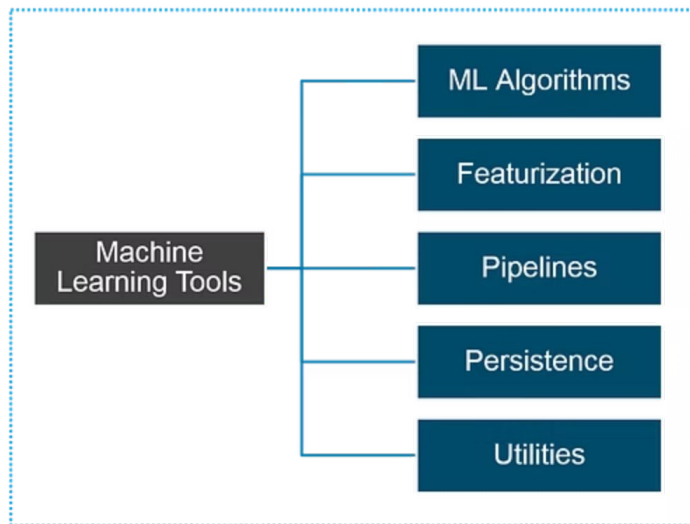
Ejemplo con regresión logística y regresión lineal.

Implementación de
algoritmos no
supervisados

Algoritmos de clustering: K-Means, Gaussian Mixture Model (GMM).

Reducción de dimensionalidad con PCA.

Introducción a machine learning escalable



Apache Spark MLlib: Machine Learning Escalable

En la actualidad, el volumen de datos que manejan las organizaciones ha crecido de forma exponencial, haciendo que los enfoques tradicionales de análisis y modelado de datos resulten insuficientes para procesar información a gran escala. Ante este desafío, surgen soluciones como **Apache Spark MLlib**, una de las bibliotecas más utilizadas para la implementación de algoritmos de **Machine Learning** de forma distribuida y escalable.

¿Qué es MLlib?

MLlib es la biblioteca de Machine Learning de Apache Spark, diseñada para trabajar con grandes volúmenes de datos de forma distribuida.

¿Por qué es relevante?

- Se integra directamente con el ecosistema de Spark
- Permite ejecutar modelos en clusters (no en una sola máquina)
- Está pensada para trabajar a escala real

¿Qué permite hacer?

- Clasificación y regresión
- Clustering
- Reducción de dimensionalidad
- Preparación de datos y evaluación de modelos

Propósito de este manual

Introducir conceptos fundamentales

Presentar los conceptos básicos de MLlib y su importancia en el ecosistema de Big Data.

Explorar características principales

Analizar las estructuras de datos y funcionalidades que hacen de MLlib una herramienta potente.

Revisar algoritmos soportados

Examinar los diferentes algoritmos de Machine Learning disponibles en la biblioteca.

Mostrar ejemplos prácticos

Presentar implementaciones de modelos supervisados y no supervisados adaptados a entornos de Big Data.

Introducción a Machine Learning Escalable

El Machine Learning tradicional enfrenta limitaciones significativas cuando se trata de procesar grandes volúmenes de datos. MLlib surge como respuesta a esta problemática, ofreciendo una solución escalable y distribuida que aprovecha toda la potencia de Apache Spark.

¿Qué es MLlib?

MLlib es la biblioteca de **Machine Learning** incluida en **Apache Spark**, diseñada para ofrecer un conjunto de herramientas que permiten construir, entrenar y evaluar modelos predictivos de forma distribuida y escalable. Esta biblioteca nació con el objetivo de solucionar uno de los principales problemas del aprendizaje automático tradicional: la dificultad de procesar y modelar grandes volúmenes de datos.

Arquitectura distribuida

1

Procesamiento paralelo

A diferencia de otras bibliotecas de Machine Learning que están pensadas para ser utilizadas en entornos de un solo equipo, MLlib fue concebida desde sus inicios para aprovechar la arquitectura distribuida de Spark.

2

Ejecución en clústeres

Los algoritmos de MLlib pueden ejecutarse de manera paralela en clústeres de cientos o miles de nodos, reduciendo los tiempos de procesamiento.

3

Manejo de datos masivos

Permite trabajar con conjuntos de datos que no caben en la memoria de una sola máquina, solucionando una limitación crítica del Machine Learning tradicional.

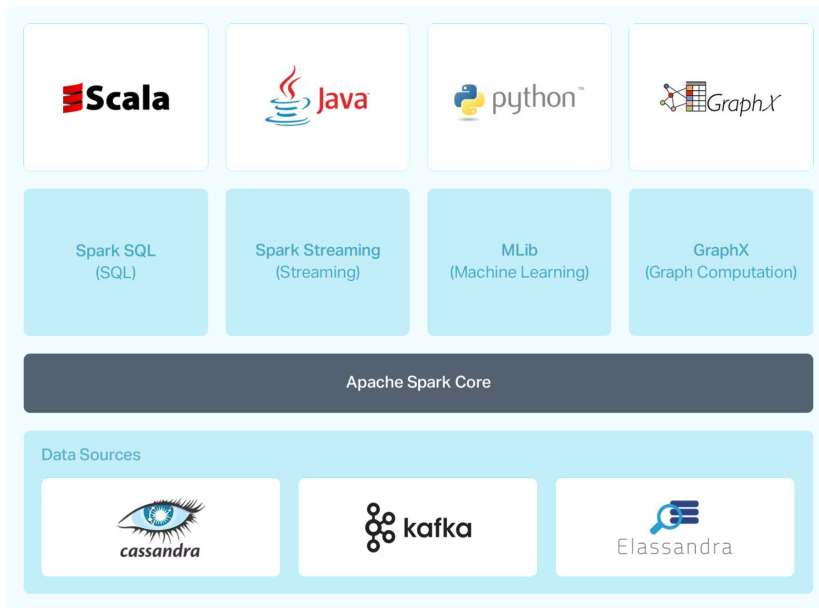
Integración con el ecosistema Spark

Uno de los aspectos más destacados de MLlib es su integración nativa con las demás librerías y componentes de Spark, como **Spark SQL** y **Spark Streaming**.

Esto permite construir flujos de trabajo completos de análisis de datos que incluyan:

- Preparación de datos
- Entrenamiento de modelos
- Predicción en tiempo real

Todo esto dentro del mismo entorno, facilitando el desarrollo de soluciones end-to-end.



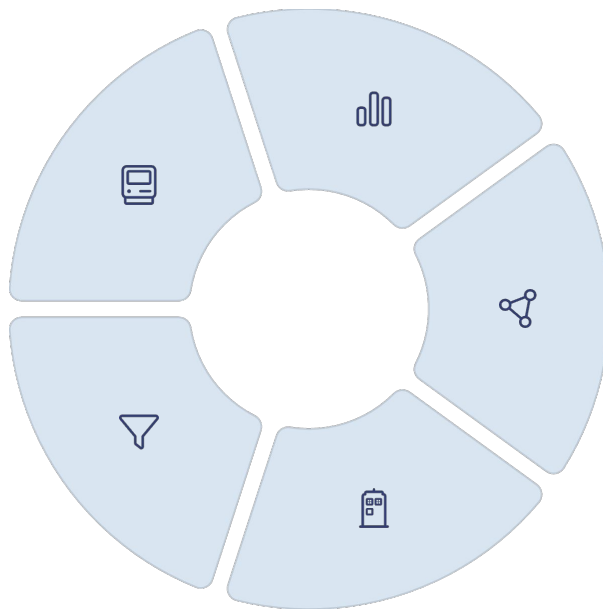
Algoritmos disponibles en MLlib

Clasificación

Algoritmos para predecir categorías o clases a partir de datos etiquetados.

Filtrado colaborativo

Algoritmos para sistemas de recomendación basados en preferencias.



Regresión

Modelos para predecir valores numéricos continuos.

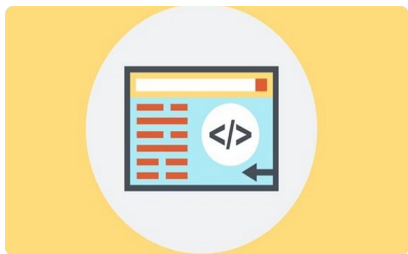
Clustering

Técnicas para agrupar datos similares sin etiquetas previas.

Reducción de dimensionalidad

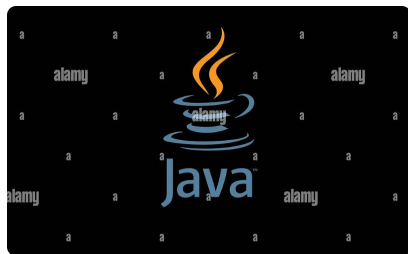
Métodos para simplificar conjuntos de datos complejos.

Soporte multilinguaje



Scala

Lenguaje nativo de Spark, ofrece la API más completa y con mejor rendimiento.



Java

Integración completa para entornos empresariales y aplicaciones existentes.



Python (PySpark)

Popular entre científicos de datos por su facilidad de uso y ecosistema de análisis.



R

Soporte para estadísticos y analistas familiarizados con este lenguaje.

Evolución de MLlib

Primeras versiones

Inicialmente basada en RDDs (Resilient Distributed Datasets), la estructura de datos fundamental de Spark.

1

2

Spark 2.0

El desarrollo se enfocó en la API basada en **DataFrames**, dejando de lado la API basada en **RDDs**.

API moderna

La nueva API proporciona un rendimiento superior y una interfaz más sencilla y expresiva para los usuarios.

3

4

MLlib Tools

Incorporación de utilidades adicionales que facilitan tareas comunes como la carga y transformación de datos, la selección de características y la evaluación de modelos.

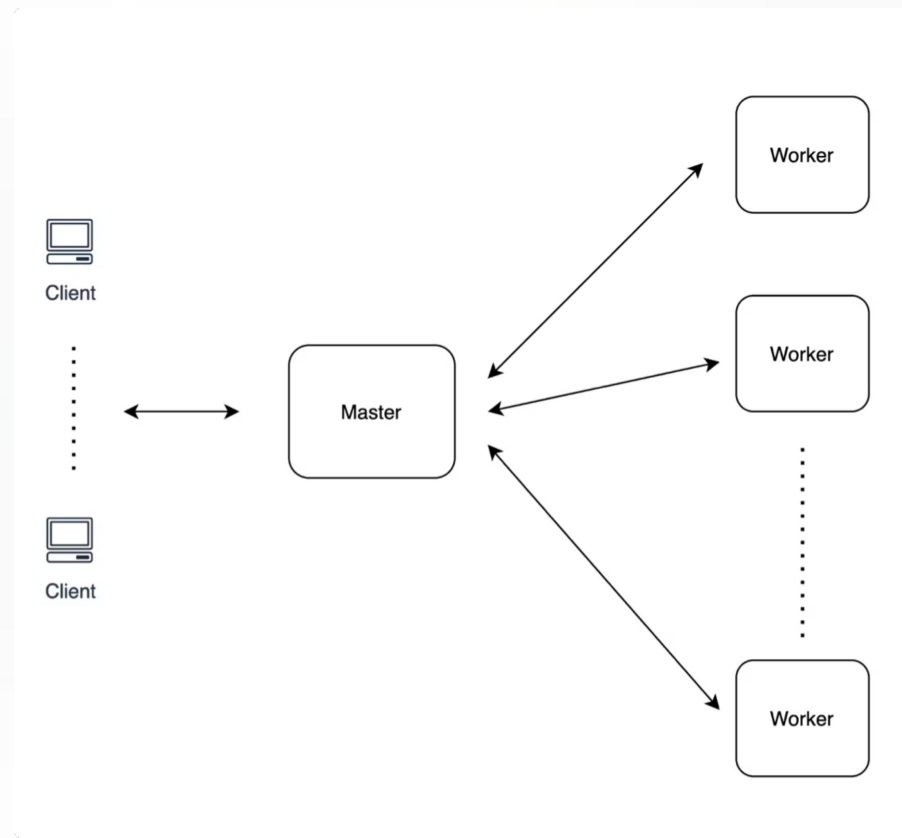
Resumen de MLlib

- ❗ MLlib es una biblioteca robusta y escalable que permite implementar soluciones de Machine Learning sobre grandes volúmenes de datos, integrándose de forma natural con el ecosistema Apache Spark y ofreciendo un rendimiento óptimo en entornos distribuidos.

Características de MLlib y MLlib Tools

La biblioteca **MLlib** de Apache Spark destaca por ofrecer un conjunto de características que la convierten en una de las herramientas más potentes para el desarrollo de modelos de Machine Learning sobre grandes volúmenes de datos.

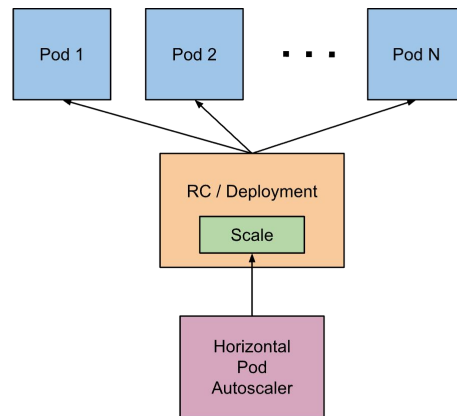
Sus ventajas permiten a los profesionales de datos afrontar los retos de la escalabilidad y el rendimiento, esenciales en contextos de Big Data.



Escalabilidad horizontal

Una de las principales características de MLlib es su **escalabilidad horizontal**. Gracias a la arquitectura distribuida de Spark, MLlib permite:

- Procesar y modelar datasets que superan la capacidad de memoria de un solo equipo
- Ejecutar algoritmos en paralelo sobre un clúster de nodos
- Reducir considerablemente los tiempos de entrenamiento y evaluación de modelos



Compatibilidad con múltiples lenguajes

Otra característica clave es la **compatibilidad con múltiples lenguajes**. MLlib ofrece APIs en **Scala, Java, Python (PySpark) y R**, lo que facilita:

Integración flexible

Incorporación de sus funcionalidades en proyectos existentes independientemente del lenguaje utilizado.

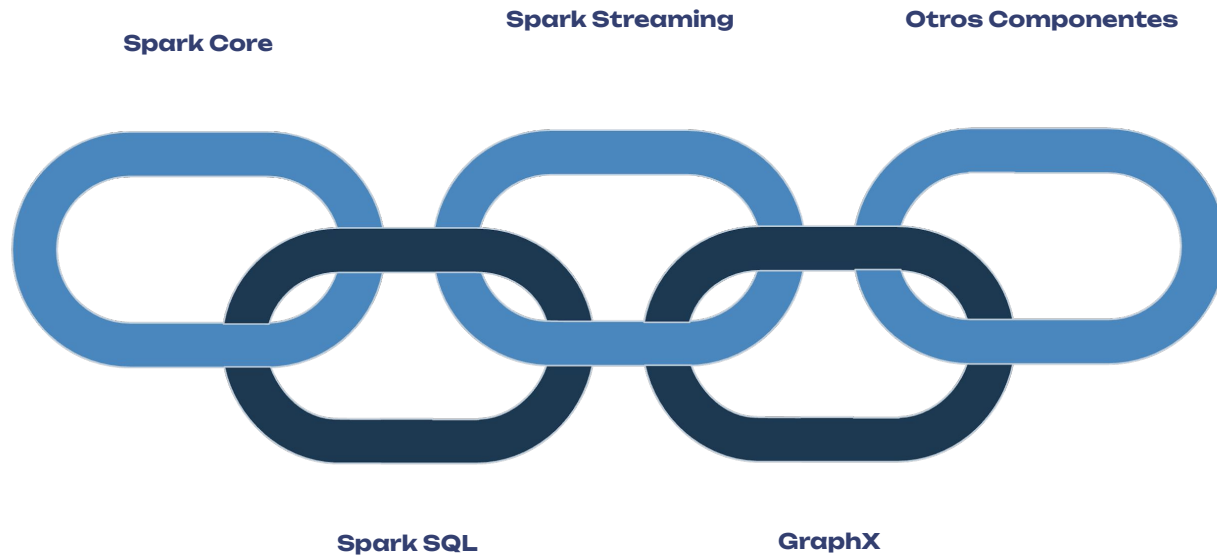
Colaboración multidisciplinaria

Facilita el trabajo entre equipos con diferentes preferencias de lenguajes de programación.

Curva de aprendizaje reducida

Los profesionales pueden utilizar el lenguaje con el que están más familiarizados.

Integración con el ecosistema Spark



MLlib se integra de forma nativa con otros módulos del ecosistema Spark, como **Spark SQL**, **Spark Streaming** y **GraphX**. Esto permite construir flujos de trabajo completos que incluyen la ingestión, transformación, análisis y visualización de datos, utilizando una única plataforma.

Catálogo de algoritmos

Aprendizaje supervisado

- Clasificación binaria y multiclase
- Regresión lineal y no lineal
- Árboles de decisión y ensambles

Aprendizaje no supervisado

- Clustering (K-means, GMM)
- Reducción de dimensionalidad (PCA)
- Detección de anomalías

Otros algoritmos

- Filtrado colaborativo
- Sistemas de recomendación
- Análisis de texto y NLP

Pipelines de Machine Learning

Una característica distintiva de MLlib es la utilización de **pipelines de Machine Learning**.

Estos pipelines permiten encadenar diferentes etapas de procesamiento de datos y modelado, facilitando la automatización y reproducibilidad de los flujos de trabajo.

MLlib Tools: Transformación y preparación de datos



Normalización

Ajuste de valores numéricos a escalas comparables para mejorar el rendimiento de los algoritmos.



Escalado

Transformación de variables para que tengan propiedades estadísticas específicas.



Codificación

Conversión de variables categóricas a formatos numéricos interpretables por los algoritmos.



Selección de características

Identificación de las variables más relevantes para el modelo, reduciendo dimensionalidad.

MLlib Tools: Evaluación de modelos

Métricas para clasificación

- Precisión (Precision)
- Exhaustividad (Recall)
- F1-score
- Área bajo la curva ROC

Métricas para regresión

- Error cuadrático medio (MSE)
- Raíz del error cuadrático medio (RMSE)
- Error absoluto medio (MAE)
- R^2 (coeficiente de determinación)

Estas métricas permiten evaluar objetivamente el rendimiento de los modelos y comparar diferentes aproximaciones.

MLlib Tools: Optimización de hiperparámetros



Definición de parámetros

Identificación de los hiperparámetros a optimizar y sus rangos de valores posibles.



Grid Search

Búsqueda exhaustiva en una cuadrícula de combinaciones de parámetros predefinidas.



Validación cruzada

Evaluación de cada combinación mediante particiones múltiples de los datos.



Selección del mejor modelo

Identificación de la combinación de parámetros que maximiza el rendimiento según las métricas elegidas.

Compatibilidad con formatos de datos

Otra ventaja de MLlib es su compatibilidad con los formatos de datos utilizados en Spark, como **DataFrames y Datasets**.

Esto facilita la manipulación y transformación de los datos antes y después del modelado.

CHANGING DATA TYPE A GUIDE TO TRANSFORMING DATA FORMATS

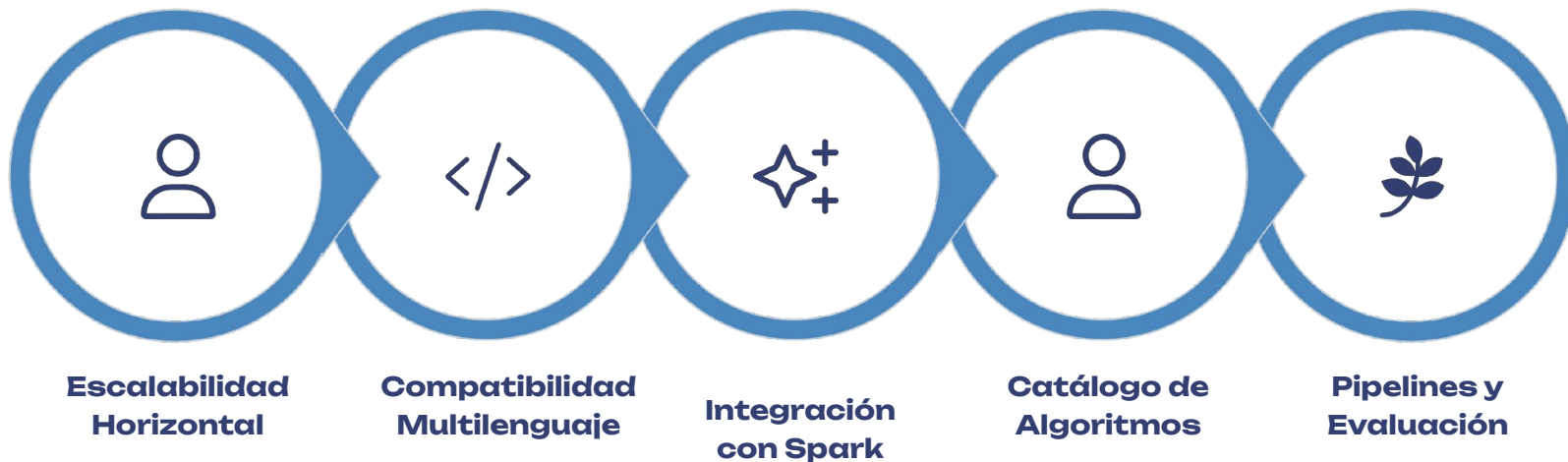
Data Type	Description
Integer	A whole number without decimal places.
Float	A number with decimal places.
String	A sequence of characters.
Boolean	Represents either true or false.
Date	Represents a specific calendar date.
Time	Represents a specific time of day.
DateTime	Represents a specific date and time.
Char	A fixed-length string.
Varchar	A variable-length string.
Text	Stores large amounts of text data.
Blob	Stores binary data, such as images or files.
Decimal	A precise decimal number with a fixed scale.
Array	A collection of values of the same data type.
JSON	Stores structured data in a human-readable format.
Enum	A set of predefined values for a column.
Object	Stores complex, unstructured data.
XML	Stores hierarchical data in a text format.
Geometric	Stores geometric shapes, such as points or polygons.
UUID	Universally unique identifier.
Interval	Represents a duration of time.
Money	Stores currency values.
Percentage	Represents a fraction out of 100.

Evolución continua

- ❏ MLlib está en constante evolución y es mantenida por la comunidad Apache Spark. Cada nueva versión incorpora mejoras de rendimiento, nuevos algoritmos y mayor compatibilidad con los estándares de la industria.

Esta evolución garantiza que MLlib se mantenga relevante y actualizada con las últimas técnicas y avances en el campo del Machine Learning.

Resumen de características



En resumen, las características de MLlib y MLlib Tools hacen de esta biblioteca una solución robusta, escalable y eficiente para implementar proyectos de Machine Learning en entornos de Big Data.

Estructuras de datos de MLlib

Para operar eficientemente sobre grandes volúmenes de datos, **MLlib** utiliza estructuras de datos optimizadas y adaptadas al procesamiento distribuido.

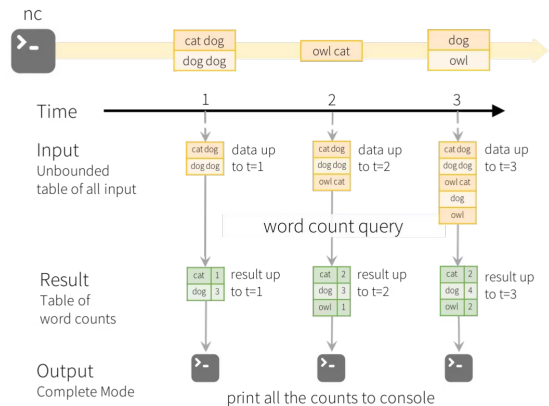
Estas estructuras permiten manejar información de forma eficiente, tanto para la preparación de los datos como para la ejecución de los algoritmos de Machine Learning.

DataFrame: estructura principal

La principal estructura de datos utilizada por MLlib es el **DataFrame**. Un DataFrame en Spark es una colección distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional.

Los DataFrames ofrecen:

- Operaciones eficientes de filtrado, agrupamiento y transformación
- Escalabilidad para grandes volúmenes de datos
- API de alto nivel



Model of the Quick Example

Uso de DataFrames en MLlib

Datos de entrada

Los DataFrames contienen los datos de entrada para los modelos de Machine Learning, incluyendo tanto las características (features) como las etiquetas (labels) asociadas a cada registro.

Transformaciones

Permiten aplicar operaciones de preparación de datos como limpieza, normalización y codificación de variables categóricas.

Resultados

Los DataFrames son utilizados para almacenar los resultados del modelado, como las predicciones y las métricas de evaluación.

Vectores en MLlib

Otra estructura fundamental en MLlib es el **Vector**. Este tipo de dato se utiliza para representar las características numéricas de cada observación de manera compacta y eficiente.

DenseVector

Utilizado cuando la mayoría de los elementos son diferentes de cero. Almacena todos los valores en un array.

Ejemplo: [1.0, 2.0, 3.0, 4.0]

SparseVector

Utilizado cuando la mayoría de los elementos son ceros.

Almacena únicamente los índices y valores de los elementos no nulos, optimizando el uso de memoria.

Ejemplo: (10, [0,3,5], [1.0,4.0,2.0]) - Vector de tamaño 10 con valores no nulos en las posiciones 0, 3 y 5.

Matrices en MLlib

Además de los vectores, MLlib utiliza estructuras como **Matrices** para representar datos bidimensionales, especialmente en algoritmos como la descomposición en valores singulares (SVD) o la factorización de matrices para sistemas de recomendación.

Al igual que con los vectores, existen implementaciones densas y dispersas para optimizar el uso de memoria según la naturaleza de los datos.



Live Coding

¿En qué consistirá la Demo?

Exploraremos conceptualmente cómo preparar un entorno en PySpark para trabajar con MLlib y ejecutar un flujo básico de Machine Learning sobre un gran volumen de datos.

1. *Crear una SparkSession con soporte para MLlib.*
2. *Cargar datos en un DataFrame y aplicar transformaciones iniciales.*
3. *Representar características como vectores y preparar datos para el modelo.*
4. *Ejecutar un algoritmo simple de clasificación o regresión.*
5. *Evaluar el modelo con métricas de MLlib.*



Ejercicio

Explorando MLlib y sus estructuras de datos

Explorando MLlib y sus estructuras de datos

Contexto: 🙌

Queremos que te familiarices con las principales estructuras y herramientas que ofrece MLlib para el trabajo con grandes volúmenes de datos.

Consigna: 📝

1. Investiga y describe los componentes clave de MLlib y su integración con Spark.
2. Explica la diferencia entre un DenseVector y un SparseVector, y cuándo conviene usar cada uno.
3. Identifica tres algoritmos supervisados y dos no supervisados disponibles en MLlib.
4. Explica el concepto de pipeline en MLlib y su utilidad.

Paso a paso: ⚙️

1. Utiliza la documentación oficial de MLlib como referencia.
2. Resume la información en un esquema o cuadro comparativo.
3. Presenta ejemplos de casos de uso para cada algoritmo.

Tiempo 🕒: 45 Minutos



Momento:

Time-out!



5 -10 min.



Introducción a machine learning escalable

Pipeline: flujos de trabajo

Definición

Un Pipeline es una secuencia de etapas que incluyen transformadores y estimadores, permitiendo encadenar operaciones de forma coherente.

Estimadores

Componentes que aprenden a partir de los datos (por ejemplo, un modelo de regresión).

Transformadores

Componentes que transforman un DataFrame en otro (por ejemplo, escalado de características).

Ejecución

Al ejecutar un pipeline, cada etapa se aplica secuencialmente, facilitando la reproducibilidad y mantenimiento del flujo de trabajo.

Model y Transformer

Model

Representa un modelo entrenado, resultado de aplicar un estimador sobre un conjunto de datos. Contiene los parámetros aprendidos durante el entrenamiento.

Transformer

Componente que aplica transformaciones sobre nuevos datos, ya sea para preprocesamiento o para realizar predicciones con un modelo entrenado.

Estas estructuras permiten separar claramente la fase de entrenamiento de la fase de aplicación del modelo, facilitando su despliegue en entornos de producción.

Resumen de estructuras de datos

DataFrame

Para la gestión de los datos de entrada y salida, organizados en columnas y filas distribuidas.

Vector (Dense y Sparse)

Para representar las características numéricas de forma eficiente según su densidad.

Matriz

Para operaciones avanzadas que requieren representación bidimensional de datos.

Pipeline

Para la construcción de flujos de trabajo completos y reproducibles.

Model y Transformer

Para la aplicación y evaluación de modelos entrenados.

Importancia de las estructuras de datos

- ① Estas estructuras forman la base sobre la cual se construyen y ejecutan los algoritmos de Machine Learning en Spark MLlib, permitiendo un procesamiento eficiente y escalable de grandes volúmenes de datos.

La elección adecuada de las estructuras de datos es fundamental para optimizar el rendimiento y la eficiencia de los modelos de Machine Learning en entornos distribuidos.

Algoritmos de Machine Learning soportados por MLlib

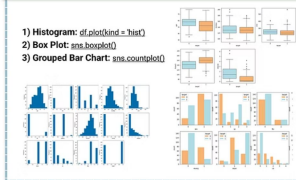
MLlib ofrece un conjunto amplio de algoritmos de **aprendizaje automático** diseñados para funcionar en entornos distribuidos y adaptados a diferentes tipos de problemas.

Estos algoritmos cubren tanto el aprendizaje supervisado como el no supervisado, permitiendo a los profesionales de datos resolver múltiples desafíos analíticos sobre grandes volúmenes de información.

Machine Learning Algorithms - Classification

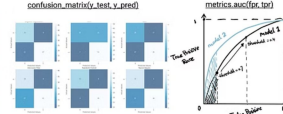
Exploratory Data Analysis (EDA)

- 1) Histogram: `df.plot(kind='hist')`
- 2) Box Plot: `ans.boxplot()`
- 3) Grouped Bar Chart: `ans.countplot()`

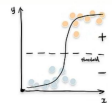


Model Evaluation

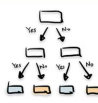
Confusion Matrix
`confusion_matrix(y_test, y_pred)`



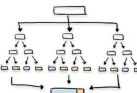
Logistic Regression



Decision Tree



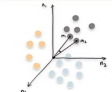
Random Forest



Support Vector Machine



K Nearest Neighbour



Naive Bayes



Algoritmos de aprendizaje supervisado: Clasificación



Regresión logística

Utilizada para problemas de clasificación binaria y multiclase, estimando la probabilidad de pertenencia a cada categoría.



Árboles de decisión

Modelos basados en reglas de decisión jerárquicas que dividen los datos según valores de características.



Random Forest

Conjunto de árboles de decisión que mejora la precisión del modelo mediante la combinación de múltiples clasificadores.



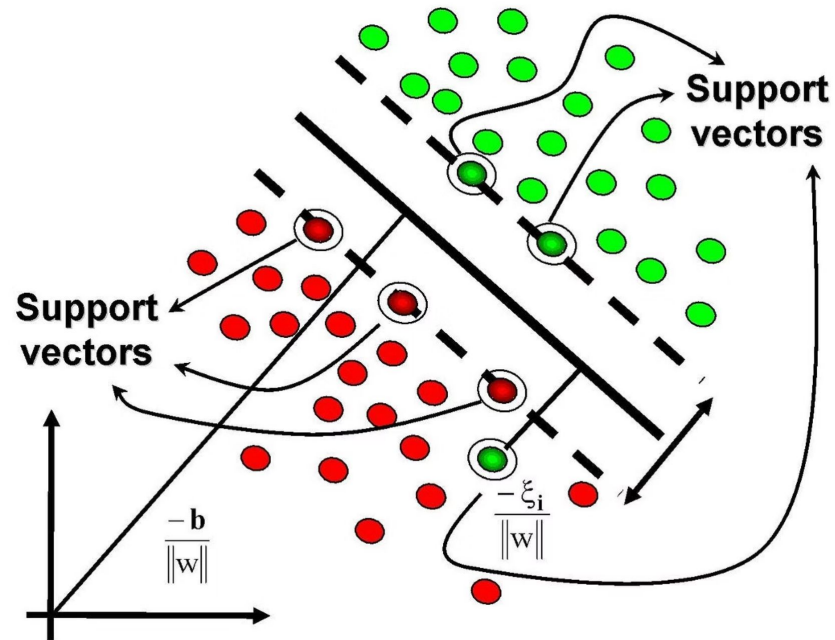
Gradient-Boosted Trees

Técnica basada en boosting para mejorar el rendimiento predictivo mediante la construcción secuencial de modelos.

Algoritmos de aprendizaje supervisado: Clasificación (continuación)

Support Vector Machines (SVM)

Algoritmo para clasificación binaria que busca el hiperplano que mejor separa las clases con el margen máximo.



Naive Bayes

Clasificador probabilístico basado en el teorema de Bayes, eficiente para problemas con muchas características.

Variantes implementadas:

- Multinomial Naive Bayes
- Bernoulli Naive Bayes
- Gaussian Naive Bayes

Algoritmos de aprendizaje supervisado: Regresión

Regresión lineal

Para problemas de predicción de valores continuos mediante la estimación de una función lineal.

Regresión con árboles de decisión

Utiliza árboles para predecir valores numéricos, capturando relaciones no lineales.

Random Forest Regressor

Conjunto de árboles de regresión que mejora la precisión y reduce el sobreajuste.

Gradient-Boosted Trees Regressor

Técnica de boosting aplicada a problemas de regresión para mejorar el rendimiento predictivo.

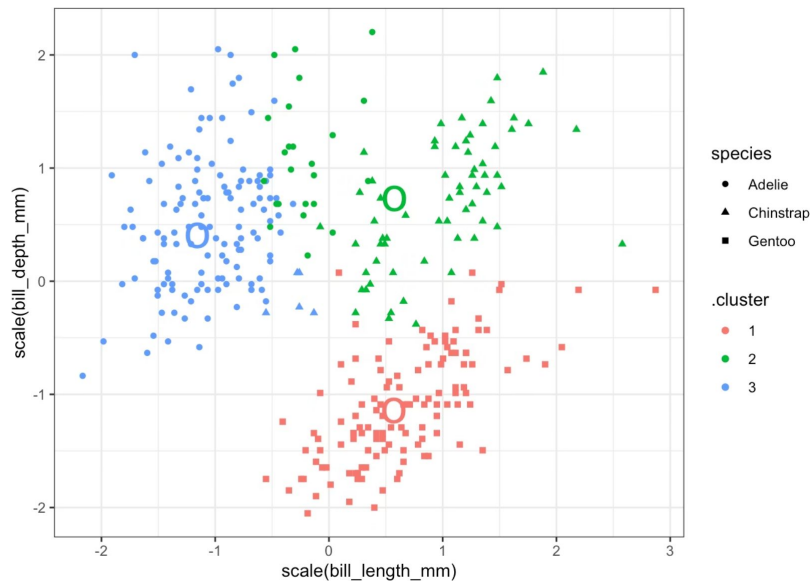
Algoritmos de aprendizaje no supervisado: Clustering

K-Means

Algoritmo para el agrupamiento de datos en un número predefinido de clústeres basándose en la similitud entre las observaciones.

Características:

- Escalable a grandes conjuntos de datos
- Implementación paralela eficiente
- Inicialización inteligente (K-means++)



Algoritmos de aprendizaje no supervisado: Clustering (continuación)



Gaussian Mixture Model (GMM)

Técnica probabilística para clustering basada en distribuciones gaussianas, útil cuando los clústeres tienen diferentes formas y tamaños.



Bisecting K-Means

Variante jerárquica del algoritmo K-Means que divide iterativamente los clústeres existentes.



Power Iteration Clustering

Algoritmo de clustering basado en la propagación de similitudes a través de un grafo de datos.

Algoritmos de aprendizaje no supervisado: Reducción de dimensionalidad

Principal Component Analysis (PCA)

Técnica para la reducción de la dimensionalidad que transforma los datos a un espacio de menor dimensión, conservando la mayor cantidad de información posible.

Beneficios:

- Reducción del ruido en los datos
- Visualización de datos multidimensionales
- Mejora del rendimiento de algoritmos posteriores

```
from pyspark.ml.feature import PCA

pca = PCA(k=2, inputCol="features", outputCol="pcaFeatures")
modelo_pca = pca.fit(data)
resultado = modelo_pca.transform(data)
resultado.select("pcaFeatures").show()
```

Algoritmos de aprendizaje no supervisado: Reducción de dimensionalidad (continuación)

Singular Value Decomposition (SVD)

Técnica de factorización matricial que descompone una matriz en tres componentes, útil para compresión de datos y eliminación de ruido.

Latent Dirichlet Allocation (LDA)

Modelo probabilístico generativo utilizado para la identificación de temas en colecciones de texto, permitiendo descubrir patrones latentes en documentos sin necesidad de supervisión.

Algoritmos de filtrado colaborativo

MLlib incluye algoritmos de **filtrado colaborativo**, utilizados principalmente en sistemas de recomendación. Uno de los más destacados es:

- ❗ **Alternating Least Squares (ALS):** Empleado para la predicción de preferencias de usuarios en plataformas como e-commerce, música o contenido audiovisual. Permite construir recomendaciones personalizadas basadas en el comportamiento pasado de usuarios similares.

Optimización para entornos distribuidos

Todos estos algoritmos han sido optimizados para funcionar sobre clústeres distribuidos, utilizando las estructuras de datos de Spark como **DataFrames** y **Vectores**. Esto permite que las tareas de entrenamiento, predicción y evaluación se realicen de forma eficiente incluso con millones de registros.

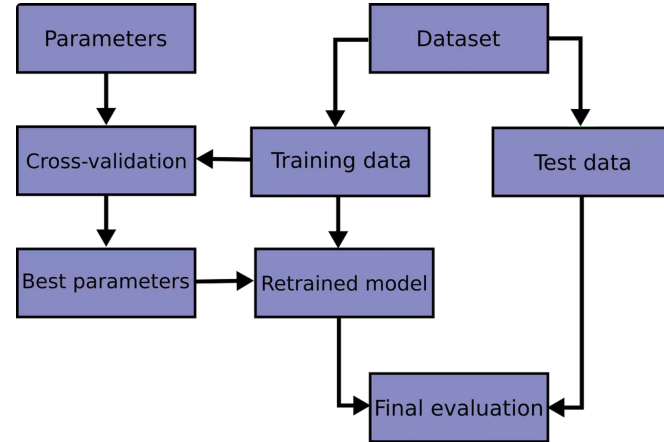
Soporte para pipelines y validación

Pipelines

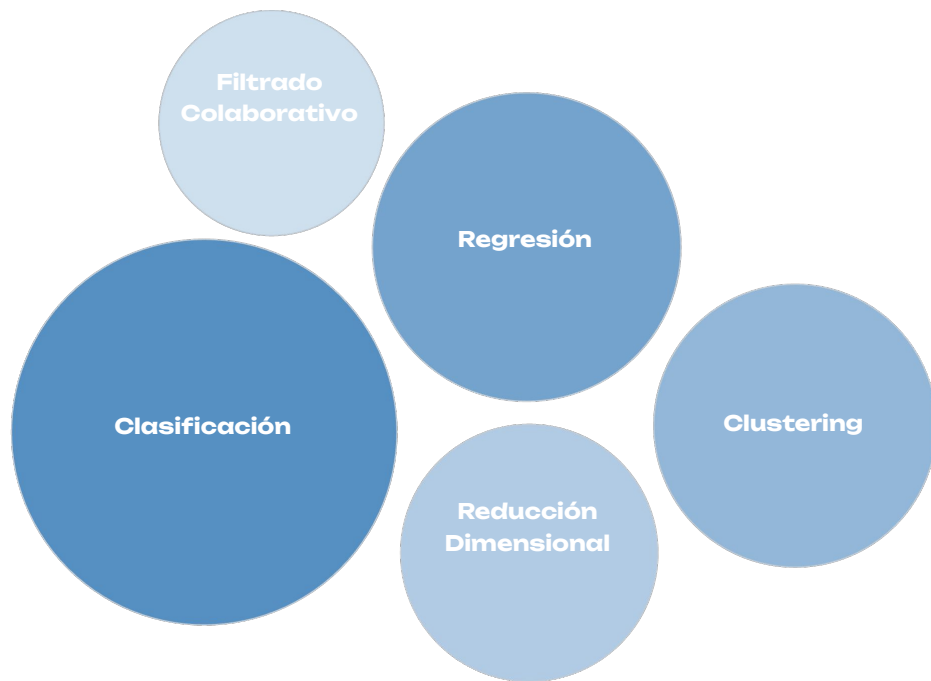
La mayoría de los algoritmos soportan la creación de **pipelines**, permitiendo encadenar las etapas de preparación de datos, entrenamiento y evaluación en un flujo automatizado.

Validación y optimización

MLlib ofrece herramientas para la **validación de modelos** y la **búsqueda de hiperparámetros**, facilitando la optimización de los algoritmos mediante técnicas como la **validación cruzada** y el **grid search**.



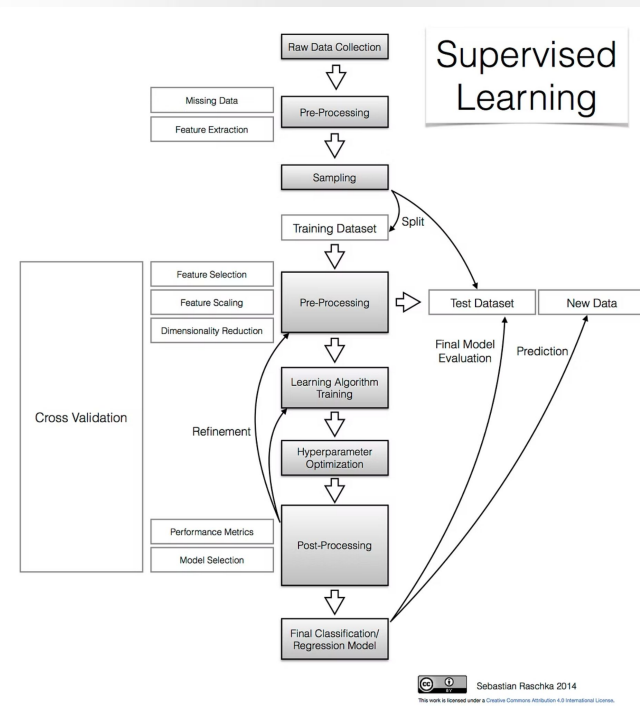
Resumen de algoritmos



En resumen, los algoritmos soportados por MLlib permiten abordar una amplia gama de problemas de Machine Learning de manera escalable, eficiente y completamente integrada con el ecosistema de Apache Spark.

Implementación de algoritmos de Machine Learning supervisados con MLlib

El uso de **MLlib** para la implementación de algoritmos supervisados permite construir modelos predictivos capaces de aprender a partir de datos etiquetados. Los problemas supervisados más comunes son la **clasificación** y la **regresión**.



Pasos para implementar algoritmos supervisados

1

Preparación de datos

Organización de los datos en un DataFrame con columnas para etiquetas y características.

2

Definición del algoritmo

Selección e instanciación del modelo adecuado para el problema.

3

División de datos

Separación en conjuntos de entrenamiento y prueba.

4

Entrenamiento

Ajuste del modelo con los datos de entrenamiento.

5

Predicción

Aplicación del modelo entrenado a nuevos datos.

6

Evaluación

Medición del rendimiento del modelo con métricas adecuadas.

Preparación de los datos

Para llevar a cabo la implementación de un algoritmo supervisado con MLlib, el primer paso es la **preparación de los datos**. Los datos deben organizarse en un **DataFrame** que contenga una columna con la etiqueta (label) y otra con un vector de características (features).

VectorAssembler

Transformador que combina múltiples columnas en un único vector de características.

StringIndexer

Convierte columnas de texto en índices numéricos, necesario para variables categóricas.

OneHotEncoder

Transforma variables categóricas en vectores binarios para su uso en algoritmos.

Normalizer/StandardScaler

Normaliza o estandariza las características numéricas para mejorar el rendimiento.

Definición del algoritmo

Para clasificación binaria

```
from pyspark.ml.classification import LogisticRegression  
  
# Crear el modelo  
  
lr = LogisticRegression( maxIter=10, regParam=0.01, elasticNetParam=0.8, labelCol="label", featuresCol="features")
```

Para problemas de regresión

```
from pyspark.ml.regression import LinearRegression  
  
# Crear el modelo  
  
lr = LinearRegression( maxIter=10, regParam=0.3, elasticNetParam=0.8, labelCol="label", featuresCol="features")
```

División de los datos

El siguiente paso consiste en **dividir los datos en conjuntos de entrenamiento y prueba** para evaluar el desempeño del modelo:

```
# Dividir los datos en conjuntos de entrenamiento (70%) y prueba (30%)
train_data, test_data = data.randomSplit([0.7, 0.3], seed=42)
print(f"Datos de entrenamiento: {train_data.count()} registros")
print(f"Datos de prueba: {test_data.count()} registros")
```

Esta división permite entrenar el modelo con un subconjunto de datos y evaluarlo con datos que no ha visto durante el entrenamiento, obteniendo así una medida más realista de su capacidad de generalización.

Entrenamiento del modelo

A continuación, se **entrena el modelo** utilizando el método `.fit()` sobre el conjunto de entrenamiento:

```
# Entrenar el modelo
model = lr.fit(train_data)

# Mostrar los coeficientes y el intercepto (para regresión)
print(f"Coeficientes: {model.coefficients}")
print(f"Intercepto: {model.intercept}")
```

Durante esta fase, el algoritmo ajusta sus parámetros internos para minimizar el error de predicción en los datos de entrenamiento.

Realización de predicciones

Una vez entrenado, se pueden realizar **predicciones** sobre el conjunto de prueba:

```
# Realizar predicciones
predictions = model.transform(test_data)

# Mostrar algunas predicciones
predictions.select("features", "label", "prediction").show(5)
```

El método transform() aplica el modelo entrenado a nuevos datos, generando una columna adicional con las predicciones.

Evaluación del modelo

Para clasificación binaria

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Evaluar el modelo
evaluator = BinaryClassificationEvaluator(labelCol="label", rawPredictionCol="rawPrediction",
metricName="areaUnderROC")

auc = evaluator.evaluate(predictions)

print(f"Área bajo la curva ROC: {auc}")
```

Para problemas de regresión

```
from pyspark.ml.evaluation import RegressionEvaluator

# Evaluar el modelo

evaluator = RegressionEvaluator( labelCol="label", predictionCol="prediction", metricName="rmse")

rmse = evaluator.evaluate(predictions)

print(f"Error cuadrático medio: {rmse}")
```


Uso de Pipelines

Una práctica recomendada es integrar todas estas etapas en un **Pipeline**, lo que permite automatizar el flujo de trabajo y facilita la reutilización y mantenimiento del modelo.

```
from pyspark.ml import Pipeline
# Crear el pipeline
pipeline = Pipeline(stages=[ vectorAssembler, # Transformador para crear el vector de características scaler,
# Opcional: normalización de características lr
# Algoritmo de aprendizaje])

# Entrenar el pipeline completo
pipeline_model = pipeline.fit(train_data)

# Aplicar el pipeline a los datos de prueba
predictions = pipeline_model.transform(test_data)
```

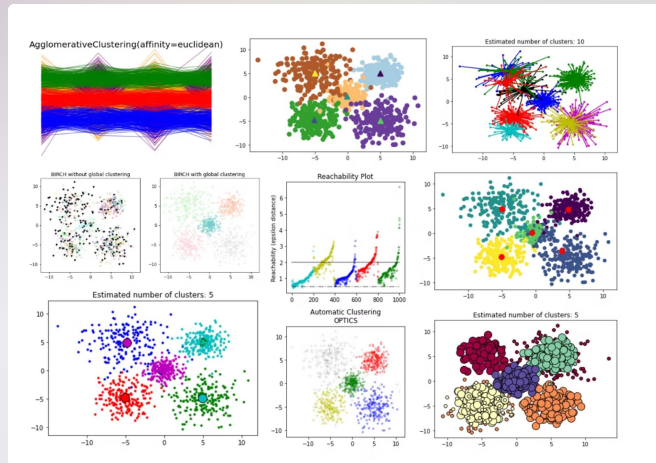
Resumen de implementación supervisada



La implementación de algoritmos supervisados con MLlib es escalable y puede aplicarse sobre grandes conjuntos de datos distribuidos, lo que la convierte en una solución ideal para entornos de Big Data.

Implementación de algoritmos de Machine Learning no supervisados con MLlib

Además de los algoritmos supervisados, **MLlib** proporciona un conjunto de herramientas para implementar algoritmos de **aprendizaje no supervisado**. Estos algoritmos permiten identificar patrones, estructuras y relaciones ocultas en los datos sin necesidad de contar con etiquetas o valores objetivo.



Implementación de K-Means

Uno de los algoritmos no supervisados más utilizados es el **K-Means**, que permite agrupar datos en un número predefinido de clústeres, basándose en la similitud entre las observaciones. La implementación de K-Means con MLlib sigue un flujo similar al de los algoritmos

```
from pyspark.ml.clustering import KMeans
# Crear una instancia del algoritmo K-Means
kmeans = KMeans( k=3, # Número de clústeres
                 seed=42, # Semilla para reproducibilidad
                 featuresCol="features",
                 predictionCol="cluster")
# Entrenar el modelo
model = kmeans.fit(data)
# Obtener los centroides de los clústeres
centers = model.clusterCenters()
print("Centroides de los clústeres:")for center in centers: print(center)
```

Predicción de clústeres y evaluación

Luego, se **entrena el modelo** utilizando el método `.fit()`:

```
# Predecir a qué clúster pertenece cada observación
predictions = model.transform(data)
# Mostrar algunas predicciones
predictions.select("features", "cluster").show(5)
```

Mllib también ofrece herramientas para **evaluar la calidad del agrupamiento**, como el **Silhouette Score**, que mide qué tan bien definidos están los clústeres:

```
from pyspark.ml.evaluation import
ClusteringEvaluator
# Evaluar el modelo
evaluator = ClusteringEvaluator(
    predictionCol="cluster", featuresCol="features",
    metricName="silhouette")
silhouette = evaluator.evaluate(predictions)
print(f"Silhouette Score: {silhouette}")
```

Gaussian Mixture Model (GMM)

Otro algoritmo no supervisado disponible en MLlib es el **Gaussian Mixture Model (GMM)**, que utiliza una combinación de distribuciones gaussianas para modelar los datos. Este enfoque es útil cuando los clústeres tienen diferentes formas y tamaños.

```
from pyspark.ml.clustering import GaussianMixture
# Crear una instancia del algoritmo GMM
gmm = GaussianMixture( k=3, # Número de componentes gaussianas
                      seed=42, # Semilla para reproducibilidad
                      featuresCol="features",
                      predictionCol="cluster")

# Entrenar el modelo
model = gmm.fit(data)
# Mostrar los pesos y parámetros de cada componente
for i in range(model.getK()):
    print(f"Peso de la componente {i}: {model.weights[i]}")

print(f"Media de la componente {i}: {model.gaussiansDF.collect()[i].mean}")
print(f"Covarianza de la componente {i}: {model.gaussiansDF.collect()[i].cov}")
```

Reducción de dimensionalidad con PCA

Además, MLlib soporta técnicas de **reducción de dimensionalidad** como el **Análisis de Componentes Principales (PCA)**.

PCA permite transformar los datos a un espacio de menor dimensión, conservando la mayor cantidad de información posible:

```
from pyspark.ml.feature import PCA
# Crear una instancia de PCA
pca = PCA( k=2, # Número de componentes principales
           inputCol="features",
           outputCol="pcaFeatures")
# Ajustar el modelo PCA
model = pca.fit(data)
# Transformar los datos al espacio de componentes principales
result = model.transform(data)
# Mostrar los resultados
result.select("pcaFeatures").show(5)
# Obtener la matriz de explicación de varianza
explained_variance = model.explainedVariance
print(f"Varianza explicada: {explained_variance}")
```

Resumen de implementación no supervisada

Estas herramientas permiten a los analistas y científicos de datos descubrir información valiosa en conjuntos de datos masivos, facilitando la toma de decisiones basada en patrones ocultos.

Live Coding

¿En qué consistirá la Demo?

Implementaremos paso a paso un modelo supervisado y uno no supervisado en Spark MLlib sobre un dataset simulado de gran tamaño.

1. Preparar los datos y transformar variables categóricas y numéricas con `StringIndexer`, `OneHotEncoder` y `VectorAssembler`.
2. Entrenar un modelo supervisado de clasificación con `LogisticRegression`.
3. Evaluar el modelo supervisado con métricas como AUC.
4. Implementar un modelo no supervisado con K-Means y evaluarlo con Silhouette Score.



Ejercicio

Construye y evalúa tu modelo MLlib

Construye y evalúa tu modelo MLlib

Contexto: 🙌

Aplicarás MLlib para construir y evaluar un modelo supervisado y uno no supervisado usando un dataset de gran tamaño.

Consigna: 📝

1. Selecciona un dataset con variables categóricas y numéricas.
2. Prepara los datos aplicando transformadores adecuados.
3. Implementa un modelo supervisado (clasificación o regresión) y evalúalo.
4. Implementa un modelo no supervisado (K-Means o PCA) y analiza sus resultados.
5. Integra todo en un Pipeline y documenta el flujo.

Paso a paso: ⚙️

1. Cargar datos y realizar preprocesamiento.
2. Dividir datos en entrenamiento y prueba.
3. Entrenar ambos modelos y evaluar métricas.
4. Comparar resultados y optimizar parámetros.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





¡Ponte a prueba!

Momento de ejercitación

*Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.*

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< ¡Muchas gracias! >

